Vibe Engineer Guide

Domina GitHub Copilot y multiplica tu productividad

IXTI Geoespacial | IA · Ciencia de Datos · Geoespacial

2026

“La IA no va a quitarte el trabajo.

Alguien que sepa usar IA. . . sí.”

— La nueva realidad del desarrollo de software

Vibe Coder vs. Vibe Engineer

La misma evolución que programador → desarrollador, pero con IA

Vibe Coder

"Le pido a la IA y acepto lo que salga"

- Prompt genérico, resultado genérico
- No define reglas ni estructura
- Cada archivo sale diferente
- Sin tests, sin validación
- Funciona... hasta que no

Vibe Engineer

"Configuro la IA para que trabaje con mis reglas"

- Define arquitectura y convenciones
- Crea instrucciones, agentes y skills
- Módulos idénticos y consistentes
- Tests y validación automáticos
- Código listo para producción

> *Un **programador** escribía código. Un **desarrollador** diseñaba sistemas.*
> *Un **vibe coder** le pide a la IA. Un **vibe engineer** diseña cómo la IA trabaja para él.*

Agenda

01 El nuevo panorama del desarrollo
02 Para quién es este curso
03 El proyecto de ejemplo
04 Arquitectura modular
05 Las 5 primitivas de Copilot
06 Instructions: Las reglas del juego
07 Prompts: Acciones con un clic
08 Agents: Roles especializados
09 Skills: Flujos complejos
10 Demo en vivo
11 Mejores prácticas
12 El futuro es ahora

El Nuevo Panorama

El desarrollo de software cambió para siempre

El mundo antes y después de la IA

Antes (2020)

- Buscar en Stack Overflow
- Copiar/pegar sin entender
- Escribir boilerplate a mano
- Configurar todo desde cero
- Inconsistencia entre módulos

4–6 horas por CRUD

Ahora (2026)

- La IA entiende tu contexto
- Genera código con tus reglas
- Módulos completos en minutos
- Tests incluidos automáticamente
- Consistencia garantizada

5–10 minutos por CRUD

Pero hay una diferencia crucial...

La misma herramienta.

Dos resultados completamente diferentes.

Un principiante con Copilot genera código que *funciona*.

Un desarrollador con Copilot **bien configurado** genera código que funciona, es consistente, tiene tests, y está listo para producción.

La brecha: Sin conocimiento vs. Con conocimiento

Principiante + Copilot

"Crea un CRUD de usuarios"

- Todo en un solo archivo
- Sin manejo de errores
- Sin validación
- HTTPException por todos lados
- Sin tests
- Cada módulo diferente

Resultado: caos escalable

Dev + Copilot configurado

"/new-crud Category name:str"

- 7 archivos organizados
- Excepciones custom
- Schemas Pydantic validados
- Async/await correcto
- Tests automáticos
- Todos los módulos idénticos

Resultado: código de producción

*La diferencia no es la herramienta. Es **quién la configura** y **qué reglas le da**.*

Para Quién Es Este Curso

Construyendo las bases correctas desde el inicio

Tu perfil

- Estás **empezando** en desarrollo de software
- Quieres aprender con las herramientas **del presente**
- Buscas ser **productivo desde el día uno**
- Quieres que tu código sea **profesional**, no solo funcional

No necesitas años de experiencia.
Necesitas las **herramientas correctas** y saber **cómo guiarlas**.

Aprenderás

- Configurar Copilot a tu medida
- Arquitectura modular
- Agentes especializados
- Deploy automatizado

Lo que vamos a lograr hoy

Definir reglas → **Crear** plantillas → **Un comando** = módulo → **Tests** automáticos → **Deploy** con un clic

> *De una idea a producción. Sin escribir código manualmente.*
> *Siguiendo estándares profesionales.*

El Proyecto de Ejemplo

API REST con FastAPI + PostgreSQL

Stack Tecnológico

Herramienta	Rol
Python 3.12	Lenguaje principal
FastAPI	Framework web async
SQLAlchemy 2.0	ORM (async)
PostgreSQL	Base de datos
Pydantic v2	Validación de datos
Docker Compose	Despliegue
pytest	Testing

3

Módulos CRUD de ejemplo

Users Products Orders

7

Archivos por módulo

Todos con la misma estructura

Arquitectura Modular

La clave para que la IA genere código consistente

Estructura de un Módulo

Todos los módulos son idénticos en forma

app/users/

`__init__.py`	vacío
`constants.py`	mensajes
`exceptions.py`	errores HTTP
`models.py`	tabla en DB
`schemas.py`	validación
`service.py`	lógica CRUD
`routes.py`	endpoints

Por qué esta estructura

Si todos los módulos tienen la **misma forma**, la IA puede:

- Aprender el patrón con un ejemplo
- Replicarlo perfectamente
- Ser consistente siempre

> *La estandarización no limita la creatividad. La **libera** para lo que realmente importa.*

Ejemplo: El modelo

SQLAlchemy 2.0 — Typed y moderno

```python
from datetime import datetime
from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column
from app.database import Base

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(255), unique=True)
    is_active: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime] = mapped_column(default_factory=datetime.utcnow)
    updated_at: Mapped[datetime] = mapped_column(default_factory=datetime.utcnow,
                                                 onupdate=datetime.utcnow)
```

Siempre `id`, `created_at`, `updated_at`. Sin excepciones.

Ejemplo: El servicio

Lógica pura, sin HTTP — separación de responsabilidades

```python
async def get_all(session: AsyncSession) -> list[User]:
    result = await session.execute(select(User))
    return list(result.scalars().all())

async def get_by_id(session: AsyncSession, user_id: int) -> User:
    user = await session.get(User, user_id)
    if not user:
        raise UserNotFoundException() # Excepcion custom, NO HTTPException
    return user

async def create(session: AsyncSession, data: UserCreate) -> User:
    user = User(**data.model_dump())
    session.add(user)
    await session.commit()
    await session.refresh(user)
    return user
```

Las 5 Primitivas de Copilot

Tu kit completo de personalización

El Mapa Completo

Se aplican automáticamente

Siempreactivos

Instructions
Reglas globales

File Instructions
Reglas por archivo

Bajodemanda

Prompts
Tareas reutilizables

Agents
Roles especializados

Skills
Flujos complejos

Tú decides cuándo usarlos

Ubicación de cada archivo

Tipo	Archivo	Ubicación	Se activa
Instructions	`copilot-instructions.md`	`.github/`	Siempre
File Instr.	`*.instructions.md`	`.github/instructions/`	Por patrón
Prompts	`*.prompt.md`	`.github/prompts/`	Comando /
Agents	`*.agent.md`	`.github/agents/`	Selector
Skills	`SKILL.md`	`.github/skills/<name>/`	Comando /

> *Todo vive en `.github/`. Se versiona con Git. Se comparte con el equipo.*
> ***Tus reglas viajan con tu código.***

Instructions

Las reglas del juego que Copilot siempre respeta

Instrucciones Globales

.github/copilot-instructions.md — el cerebro del proyecto

```
# Project Guidelines

## Architecture
This is a FastAPI REST API with PostgreSQL.
Each entity lives in its own package under app/.

## Conventions
- All routers use APIRouter with prefix and tags
- Standard CRUD: GET /, GET /{id}, POST /, PUT /{id}, DELETE /{id}
- Return Pydantic schemas, never raw ORM objects
- Use module-level custom exceptions, not HTTPException

## Build and Test
  pytest tests/ -v
  docker compose up --build
```

Resultado: Copilot **siempre** sigue estas reglas. No tienes que repetirlas en cada conversación.

Instrucciones por Archivo

Se activan solo cuando tocas archivos que coinciden

python-api

```
---
description: "Use when writing Python files"
applyTo: "**/*.py"
---
```

- ✔ Type hints obligatorios
- ✔ Async/await para DB
- ✔ SQLAlchemy 2.0 style
- ✔ Orden de imports

testing

```
---
description: "Use when writing test files"
applyTo: "tests/**/*.py"
---
```

- ✔ pytest + asyncio
- ✔ httpx AsyncClient
- ✔ 6 tests por CRUD
- ✔ Naming convention

Las reglas de testing **no se cargan** cuando editas código de la app, y viceversa. Contexto limpio.

Prompts

Tareas reutilizables con un solo comando

Qué es un Prompt

Una receta que ejecutas desde el chat con /

```
---
description: "Generate a complete CRUD module"
agent: "agent"
argument-hint: "Entity name and fields"
---
Generate a complete CRUD module for the entity described below.
Create all 7 files under app/<entity_name>/.
Follow the patterns from app/users/ as reference.
```

Escribes: `/new-crud`
"Category con name:str"

→

7 archivos generados
+ registro en `main.py`

Nuestros 4 Prompts

/new-crud

➕ Genera módulo CRUD completo
7 archivos + registro en main.py

Ejemplo: "Product con name, price, sku"

/run-tests

▶ Ejecuta pytest completo
Reporta passed/failed

Instala dependencias si faltan

/generate-tests

✎ Crea tests para un módulo
6 tests estándar por CRUD

Ejemplo: "users"

/deploy

🐳 Docker Compose build + up
Verifica health checks

Prueba el endpoint de salud

Agents

Personalidades con roles y herramientas limitadas

Por qué limitar herramientas

Porque con un gran poder viene una gran responsabilidad

api-developer

Herramientas: read, edit, search, execute

- ✅ Crea módulos y endpoints
- ✅ Edita código de la app
- ✅ Ejecuta comandos
- ✅ Sigue la arquitectura modular

tester

Herramientas: read, search, execute

- ✅ Escribe tests
- ✅ Corre pytest
- 🚫 **NO edita código de la app**
- ✅ Reporta fallos

> *Si un test falla, el agente `tester` arregla el **test**, no la app. Eso es separación de responsabilidades.*

Anatomía de un Agente

```
---
description: "Use when building API endpoints, creating modules..."
tools: [read, edit, search, execute]
---
You are an expert FastAPI backend developer.

## Constraints
- ALWAYS follow the modular architecture
- NEVER use HTTPException directly in services
- NEVER return raw ORM objects from routes
- ALWAYS use AsyncSession for database operations

## Workflow
1. Read existing modules to understand patterns
2. Follow the exact same structure for new code
3. Register new routers in main.py
```

El `description` es lo más importante — es cómo otros agentes lo encuentran.

Skills

Flujos complejos con plantillas y referencias

Skill vs Prompt

Ambos aparecen con / pero son muy diferentes

Prompt

- Un archivo `.prompt.md`
- Una tarea simple
- Instrucciones inline
- Sin archivos de soporte

"Genera tests para users"

Skill

- Una **carpeta** con `SKILL.md`
- Flujo multi-paso
- Plantillas y referencias
- Carga progresiva

"Genera módulo + tests + registro"

Regla: Tiene plantilla o script → **Skill**. Solo texto → **Prompt**.

Nuestro Skill: CRUD Generator

5 pasos automatizados con plantillas de referencia

```
.github/skills/crud-generator/
SKILL.md    references/templates.md    references/test-patterns.md
```

- `SKILL.md` — Instrucciones del flujo, descripción y variables de entrada
- `templates.md` — Plantillas de código para los 7 archivos del módulo
- `test-patterns.md` — Patrones estándar de testing (6 tests por CRUD)

Demo en Vivo

De cero a producción en minutos

Demo: El Flujo Completo

1. Mostrar la API funcionando: `docker compose up`
2. Mostrar `copilot-instructions.md` y explicar cada sección
3. Ejecutar `/new-crud`: "Category con name y description"
4. Ejecutar `/generate-tests`: "categories"
5. Ejecutar `/run-tests` y ver resultados en verde

Nuevo módulo. Con tests. Listo para producción. Sin escribir una línea.

Mejores Prácticas

Lo que separa a un usuario casual de un profesional

Tips para Instrucciones Efectivas

✔ Hacer

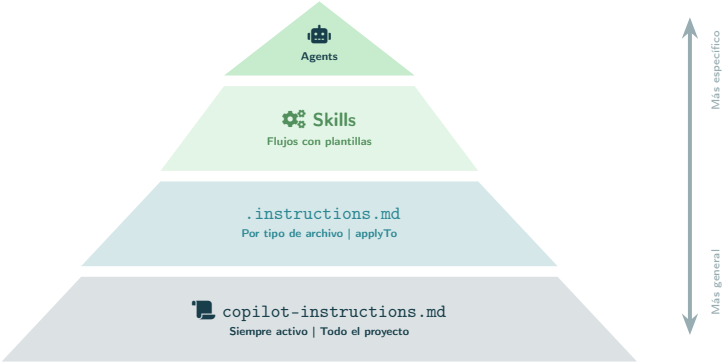
- ✔ Ser específico y conciso
- ✔ Incluir ejemplos de código
- ✔ Un archivo = una preocupación
- ✔ Keywords en `description`
- ✔ Actualizar con tu proyecto
- ✔ Versionar en Git

✘ Evitar

- ✘ Instrucciones vagas
- ✘ Todo en un solo archivo
- ✘ Repetir lo que un linter ya hace
- ✘ `applyTo: "**"` sin necesidad
- ✘ Descripciones genéricas
- ✘ Copiar el README entero

> *El `description` es la puerta de entrada. Si las palabras clave no están ahí, el agente nunca encontrará tu instrucción.*

La Pirámide de Personalización



El Futuro Es Ahora

Estás listo para lo que viene

En 2020, saber programar era la ventaja.
En 2026, saber *guiar a la IA* es la ventaja.

Los desarrolladores que dominen estas herramientas no serán reemplazados. Serán **irremplazables**.

El panorama que viene

- **Agentes autónomos** que planifican, ejecutan y validan
- **Multi-agentes** que se delegan trabajo entre sí
- **Context awareness** — la IA entiende TODO tu proyecto
- **Generación end-to-end** — de idea a deploy
- **Custom workflows** — cada equipo con su propio flujo

*La pregunta no es si vas a usar IA para programar. Es **cuándo vas a empezar a usarla bien**.*

2022
Autocompletado básico

2024
Chat + agentes simples

2026 ←
Custom agents + skills

2027+
Agentes autónomos

Reflexión final

Hace 10 años, aprender a programar requería meses de estudio antes de crear algo útil.

Hoy, con las herramientas correctas, **puedes ser productivo desde el día uno**.

La barrera de entrada bajó.
Pero la barrera de la *excelencia* sigue ahí.

Este curso es tu atajo.

Ejercicio Práctico

Tu turno de crear algo

Ejercicio: Crea tu Propio Módulo

Tu misión

Usar las herramientas del curso para crear un módulo de **"Categories"** desde cero.

1. Abre el chat de Copilot en VS Code
2. Ejecuta `/new-crud` con: *"Category con name:str, description:str|None, is_active:bool"*
3. Verifica los 7 archivos en `app/categories/`
4. Ejecuta `/generate-tests` con: *"categories"*
5. Ejecuta `/run-tests` para validar
6. Abre `localhost:8000/docs` y prueba los endpoints

Bonus: Agrega una regla nueva a `copilot-instructions.md` y comprueba que Copilot la sigue en la siguiente generación.

Conclusión

Lo que te llevas hoy

Resumen Ejecutivo

Primitiva	Función	Analogía
Instructions	Reglas globales del proyecto	La constitución
File Instructions	Reglas por tipo de archivo	Leyes específicas
Prompts	Tareas con un clic	Recetas de cocina
Agents	Roles especializados	Empleados expertos
Skills	Flujos con plantillas	Manuales de procedimiento

> *La IA no reemplaza al desarrollador. **Amplifica** al desarrollador que sabe guiarla.*

Preguntas

El código del curso está en el repositorio.
Experimenta, rompe cosas, aprende.

Innovación Geoespacial

IA · Ciencia de Datos · Geoespacial

© 2026 IXTI Geoespacial. Todos los derechos reservados.